

Real-Time Ad-Tech Traffic Analyzer: Bot Detection Using HyperLogLog++ and Streaming Analytics

Aseel Omar

Faculty of Graduate Studies - Artificial Intelligence
An-Najah National University
Jenin, Palestine
Email: s12356791@stu.najah.edu

Abstract—Digital advertising platforms handle millions of ad events per second and are extremely vulnerable to click farms and automated bots. The main challenge in identifying these attacks in real time is the cost of performing exact counting of unique users in high-velocity streams to detect abnormal repetition. This work develops a real-time streaming analytics system using Apache Kafka, Spark Structured Streaming, and MongoDB for bot traffic detection. Across this pipeline, the HyperLogLog++ (HLL++)-based sketch was implemented to approximate unique user counts with $O(1)$ memory complexity, thereby overcoming the limitations of exact counting that cause scalability issues. The efficacy of the system was validated by injecting synthetic bot attacks within this pipeline, and the system can calculate “Hits-Per-User” ratios and trigger critical alerts within seconds when this anomaly is detected on a live dashboard.

Index Terms—Bot detection, ad fraud, stream processing, HyperLogLog, cardinality estimation, real-time analytics, Apache Spark, Kafka

I. INTRODUCTION

Engagement metrics validity is a cornerstone of the digital advertising ecosystem; advertisers compensate publishers for clicks and impressions, presuming that these events are driven by real users. Yet, ad fraud remains a common menace—traffic is artificially boosted by botnets. According to Juniper’s market research, if it is not dealt with, ad fraud losses could amount to \$172 billion annually by 2028 [1], disturbing analytics and sinking into marketing accounts.

The “Distinct Count Problem” constitutes a notable technical obstacle in recognizing this problem. To differentiate a real user from a bot, the system must assess the ratio of total clicks to unique users (IP addresses) within microseconds. If a stream is tight, each unique IP should be set in a hash to perform the exact count; hence, the memory of $O(N)$ grows linearly [2]. Under traffic pressure, memory disaster, and system latency are inevitable; therefore, real-time detection is impossible.

As an ad hoc alternative, batch solutions analyze traffic logs hours after the event [3]. Although it is precise, this time lag allows the attackers to suck the budget before the fraud is recognized. Therefore, real-time solutions are mandatory; the difficulty lies in finding the balance between accuracy, latency, and consumption.

To solve the scalability problem, the project implements a streaming analytics pipeline employing Probabilistic Data Structures, the HyperLogLog++ (HLL++) algorithm [4]. By sacrificing a tiny accuracy for rocketing performance, HLL++ lets the system estimate the number of unique users at fixed memory complexity; one could track millions with minimum footprints.

This paper provides a full-stack fraud detection pipeline. It elaborates integration of Apache Kafka [5] for high-throughput buffering, Spark Structured Streaming [6] for stateful aggregation, and MongoDB for document storage. Finally, this paper demonstrates the system’s performance under a simulated bot attack to verify its capability of detecting high-frequency IP repetition in real-time.

II. METHODOLOGY

A. Synthetic Data Generation Strategy

To validate the system’s fraud detection capabilities under realistic conditions, a probabilistic data generation model was designed. Unlike simple random generation, This approach simulates specific market characteristics to stress-test the pipeline’s aggregation logic.

Geographic Weighting: The generator implements a weighted probability distribution for the `country` field to mirror global ad market shares. The distribution is explicitly defined as: United States (35%), China (12%), United Kingdom (9%), Germany (7%), Palestine (6%), France (5%), Japan (5%), and Canada (4%), with the remaining 17% distributed among other regions. This ensures the system can handle non-uniform grouping keys during aggregation.

Dual-Modality Traffic Patterns: Traffic is generated using two distinct behavioral models:

- **Legitimate User Model (70%):** Events are sourced from a broad pool of 200 distinct IP addresses with low repetition rates (1-2 clicks per window), simulating organic engagement.
- **Botnet Model (30%):** Events are sourced from a restricted pool of 20 IP addresses with high frequency (10-50 clicks per window). This high-repetition pattern allows us to verify if the HLL++ algorithm correctly

identifies the low cardinality of the attacker pool despite the high volume of events.

B. Probabilistic Counting (HLL++)

Apache Spark 3.4.2 constitutes the backbone of analysis. Instead of the costly `count (DISTINCT ip_address)`, which requires gathering all information after shuffling it across the cluster, the `approx_count_distinct` function was utilized.

This implements the HyperLogLog++ algorithm. HLL++ estimates cardinality by hashing elements and identifying the greatest number of leading zeroes in the binary representation of the hash. The relative error is determined by the number of registers, m , according to the formula for standard error $1.04/\sqrt{m}$, which allows the system to identify unique users within a streaming window based on seemingly constant space complexity.

C. Detection Algorithm

Events are processed by the detection mechanism in 10-second tumbling windows. For each window, the mechanism calculates Suspicion Ratio and denotes it R :

$$R = \frac{\text{Total Hits (Count)}}{\text{Unique Users (HLL++ Estimate)}} \quad (1)$$

Thresholds are defined in the Spark logic (`AdTechTrafficAnalyzer.scala`) to categorize traffic:

- **Normal** ($R < 1.5$): Typical user behavior.
- **Suspicious** ($1.5 \leq R \leq 5.0$): Unusually high engagement.
- **Critical** ($R > 5.0$): Bot attack confirmed.

III. SYSTEM ARCHITECTURE

The platform follows a decoupled “Source-Process-Sink” architecture designed for high-throughput, fault-tolerant stream processing. The system is composed of four primary layers: Data Generation (Source), Ingestion, Processing, and Storage/Serving. Figure 1 illustrates the end-to-end data flow.

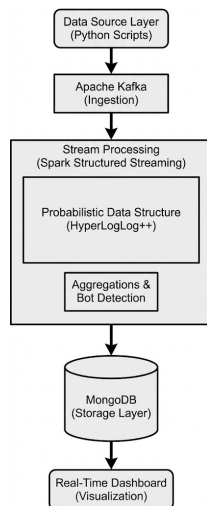


Figure 1. End-to-End Streaming Pipeline Architecture

A. Data Generation Layer

The pipeline is fed by a custom Python-based traffic generator (`ad_traffic_generator.py`) that acts as the primary data producer. Utilizing the `Faker` library, this component synthesizes a continuous stream of JSON-formatted ad events. The generator is configured to serialize these objects and push them directly to the ingestion layer with configurable throughput rates (defaulting to 50 events/second).

The generator is configured to emit events with a specific schema:

- `event_id`: UUID v4 for unique identification.
- `timestamp`: ISO 8601 formatted timestamp (synchronized to a fixed timezone, e.g., Gaza Time).
- `ip_address`: IPv4 address of the user.
- `campaign_id`: Identifier for the ad campaign (e.g., `campaign_1`).
- `action`: User interaction type (e.g., `click`, `view`).
- `country`: Geographic origin code (generated using a weighted probability distribution to simulate realistic market shares, e.g., US: 35%, CN: 12%, PS: 6%).
- `device`: Device type (e.g., `mobile`, `desktop`).

Additionally, traffic distribution logic discussed in Section 2 is implemented to mimic both legitimate and malicious behavior.

The generator pushes these events directly to the Apache Kafka broker, acting as the primary data producer for the streaming architecture.

B. Ingestion Layer (Kafka)

The entry point for the streaming architecture is an Apache Kafka cluster. A dedicated topic, `ad_stream`, serves as the high-throughput buffer between the generator and the processing engine. Kafka’s partitioned log structure ensures that the system can handle bursty traffic loads, such as those seen during the simulated bot attacks, without data loss or back-pressure impacting the generator.

C. Processing Layer (Spark Structured Streaming)

The core analytical engine is built on Apache Spark Structured Streaming. The application subscribes to the Kafka topic and processes the raw JSON payload in micro-batches (triggered every 2 seconds). Key processing tasks performed in this layer include:

- **Schema Enforcement**: Applying a strict schema to the raw JSON data to ensure type safety.
- **Watermarking**: A 30-second watermark is applied to handle late-arriving data, ensuring robust windowed aggregation even in the presence of network latency.
- **Stateful Aggregation**: The engine computes the HLL++ sketches and hit counts over 10-second tumbling windows, maintaining the state necessary to detect anomalies across time.

D. Storage and Serving Layer (MongoDB)

The processed metrics are stored in the MongoDB database (adtech), which acts as a service layer for the dashboard. The schema design is document-oriented, supporting five distinct collections to enable multi-dimensional analysis: `traffic_metrics` (global stats), `campaign_metrics` (per-campaign stats), `country_metrics` (geo-stats), `data_quality` (error logs), and `alerts` (high-severity incidents). This separation allows for efficient querying by the visualization layer.

E. Visualization Layer (Streamlit)

The final layer is a real-time dashboard built with the Streamlit framework, chosen for its ability to render interactive data applications directly from Python scripts. The dashboard (`dashboard.py`) functions as a real-time polling client that queries the MongoDB storage layer at 2-second intervals to visualize the state of the ad traffic. It implements the sigmoid-based risk scoring logic to translate raw metrics into user-friendly risk indicators (e.g., a Red/Green gauge).

The visualization logic focuses on four critical monitoring areas:

- **Traffic Divergence Visualization:** A time-series area chart compares the volume of raw events (`total_hits`) against the estimated unique users (`unique_users`). Under normal conditions, these metrics track closely ($R \approx 1.0$). A visual divergence, where event volume spikes while user counts remain flat, serves as the primary visual indicator of a bot attack.
- **Probabilistic Risk Scoring:** To quantify the threat level, the raw hits-per-user ratio (R) is transformed into a percentage probability using a sigmoid function. This normalizes the unbounded ratio into a 0-100% risk scale, driving a gauge chart that transitions from green (Safe) to red (Critical) dynamically.

$$P(bot) = \frac{1}{1 + e^{-6(R_{norm} - 0.5)}} \times 100 \quad (2)$$

where R_{norm} is the ratio normalized between the baseline (1.5) and the saturation threshold (8.0).

- **Operational Alert Feed:** A dedicated panel filters and displays documents from the `alerts` collection, highlighting incidents with “CRITICAL” or “HIGH_RISK” severity levels alongside their timestamps.
- **Data Quality Health Check:** A status indicator monitors the integrity of the ingestion pipeline by calculating the percentage of malformed events (e.g., missing IPs) in the `data_quality` collection. Thresholds are defined at <2% (Healthy), 2-5% (Degraded), and >5% (Unhealthy).

IV. DEMONSTRATION SCENARIOS

To measure the system’s usefulness, several real-time simulations were conducted, covering three separate operation scenarios: (1) baseline normal traffic, (2) concentrated bot attack on a specific campaign, and (3) degradation of data quality.

The experiments utilized synthetic traffic generated at a rate of 50 events per second (EPS). The traffic composition was configured as follows:

- **Legitimate Traffic (70%):** Sourced from a pool of 200 IP addresses with 1-2 events per user per window.
- **Bot Traffic (30%):** Sourced from a restricted pool of 20 IP addresses exhibiting high repetition rates.
- **Malformed Data (5%):** Events containing null IP addresses or invalid timestamps.

The streaming pipeline was set with a tumbling window of 10 seconds, a late data watermark of 30 seconds, and a micro-batch trigger of 2 seconds. The HyperLogLog++ algorithm used 16,384 registers (standard Spark precision). The suspicion threshold was set at the Hits-Per-User ratio (R) > 2.5.

A. Scenario 1: Normal Traffic Baseline

In the first scenario, the baseline statistics for legitimate traffic signatures were obtained by running the generator with the standard 50 EPS.

Observations: Under these settings, the dashboard showed a Hits/User ratio greater than 1.5 and less than 2.0, consistent with normal behavior from organic users. The probability of being a bot was estimated to be under 15%, which controlled the severity to “LOW RISK”. The total unique users estimated (HLL++) followed closely to total hits, as shown in Figure 2. For example, for about 500 events in a 10-second window, 320 unique IPs are estimated to appear as well, which yields a ratio of $R \approx 1.56$, classified as normal.

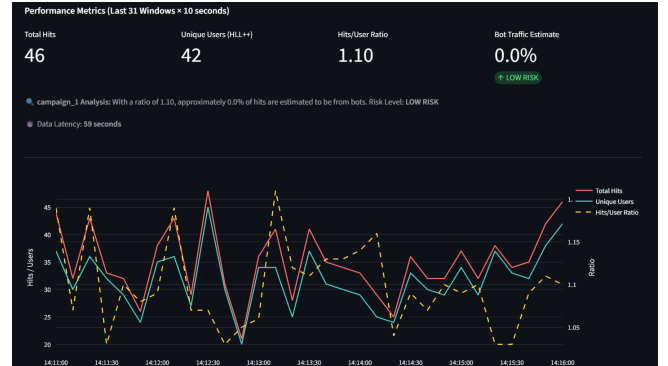


Figure 2. Dashboard displaying normal traffic baseline (Scenario 1): Total Hits = 46, Unique Users (HLL++) = 42, Hits/User Ratio = 1.10, Bot Traffic Estimate = 0.0%, Risk Level = LOW RISK. The time-series chart shows stable traffic patterns with hits and users tracking closely together.

The time-series visualization (Figure 3) confirmed stable traffic patterns with minimal variance between the volume of hits and the count of unique users.

B. Scenario 2: Concentrated Bot Attack

In order to demonstrate real-time detection, a targeted bot attack was simulated by attacking a particular campaign (“campaign_1”). The background traffic was maintained, while fake clicks were injected using this command setup:

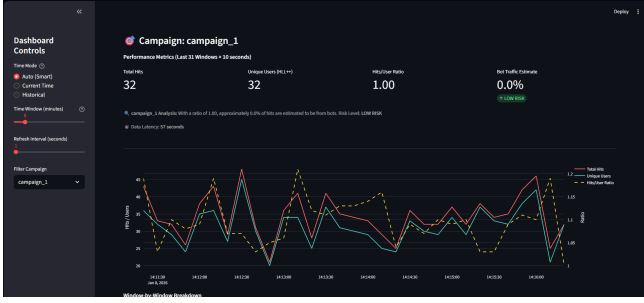


Figure 3. Time-series visualization during normal traffic: Red line (Total Hits), cyan line (Unique Users HLL++), and yellow dashed line (Hits/User Ratio) all show organic variance around baseline values ($R \approx 1.0$ -1.2). No sustained spikes indicate healthy traffic patterns.

```
python demo_bot_attack_REAL.py --duration 60 --rate 100 --campaign campaign_1
```

Listing 1. Attack Injection Command

This injected 100 clicks/second from a single IP ('192.168.1.666') for 60 seconds.

Observations: Within 15 seconds of starting, the system found anomalies on a large scale. The global metrics registered a rise in the number of hits from ~ 500 to $\sim 1,500$ per window, while the unique users stayed at ~ 320 . This led to a jump in the global Hits/User ratio to 4.7, and the dashboard triggered a "HIGH" severity alert.

The insights for "campaign_1" showed a large breach. Total hits per window went up from 50 to 1,042, while the total unique users went up by only 1 (a single bot IP). This gave a critical ratio of 27.42 and a bot probability of 95.3%.



Figure 4. Campaign_1 dashboard during concentrated bot attack (Scenario 2): Dramatic spike visible at 14:23:30 where Total Hits jumps to 1,042 while Unique Users remains at 38, resulting in Ratio = 27.42 (CRITICAL severity) and Bot Traffic Estimate = 95.3%. The sharp vertical ascent in the red line (Total Hits) contrasts with the flat cyan line (Unique Users), confirming a single-IP attack pattern.

Figure 4 shows this separation visually, while the alert log (Figure 5) shows multiple "CRITICAL" alerts as generated right after the breach threshold. This shows the correctness of HLL++ in the estimation of cardinality, since the drastic increase in clicks did not lead to an estimated rise in total unique users.

C. Scenario 3: Data Quality Degradation

The final scenario tested the pipeline's robustness against malformed data. The `inject_quality_spike.py`



Figure 5. Geographic distribution and real-time alerts panel: Left panel shows traffic predominantly from country 'XX' (3,000 hits), indicating bot origin. The right panel displays 5 recent CRITICAL and HIGH_RISK alerts for campaign_1 and GLOBAL, with ratios ranging from 5.30 to 33.32. Timestamps show alerts generated within seconds of threshold breach.

script was executed to introduce a burst of 1,000 events where 50% of the records contained null IP addresses.

Observations: The Data Quality monitor immediately reflected the injection. The percentage of malformed events surged from a baseline of $\sim 1.1\%$ to 4.96% within the affected window, changing the system status to "DEGRADED" (Figure 6).

Crucially, the bot detection logic remained unaffected. The HLL++ algorithm automatically filtered null values, ensuring that the cardinality estimation remained accurate for the valid portion of the traffic. Once the injection ceased, the sliding window mechanism ensured that the quality metrics returned to healthy levels within 60 seconds (Figure 7).

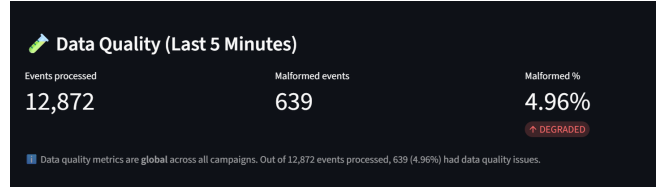


Figure 6. Data Quality monitoring during injection spike (Scenario 3): Out of 12,872 events processed, 639 (4.96%) were malformed, triggering DEGRADED status indicator (red). The system correctly identifies and isolates problematic events while maintaining accurate bot detection on valid events.

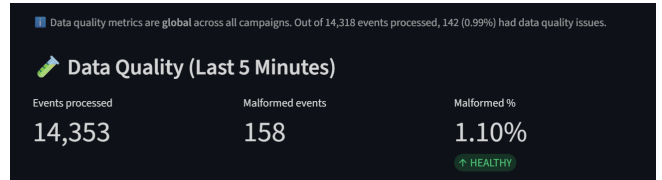


Figure 7. Data Quality after injection cleared (Scenario 3 recovery): Malformed rate returns to baseline 1.10% (158 out of 14,353 events), status indicator changes to green HEALTHY. This demonstrates the sliding window's self-healing property—old bad data naturally expires from analysis windows.

V. CONCLUSION

This project successfully demonstrates the importance of probabilistic data structures in the immediate detection of large-scale fraud. Using HyperLogLog++ within the Spark Structured Streaming framework, automated programs (bots)

were detected rapidly without the linear memory costs associated with traditional counting.

The current application relies on frequency analysis and is effective against simple flood attacks. However, HyperLogLog++ is an approximate algorithm with a standard error of approximately 1–2%, which may be a limitation for accurate billing applications. Future work will focus on integrating the Spark MLlib library to implement unsupervised learning (such as K-Means clustering) for detecting more complex distributed botnets that may bypass simple frequency thresholds.

REFERENCES

- [1] Juniper Research, “Ad fraud: Trends, challenges & market forecasts 2024-2028,” tech. rep., Juniper Research Ltd, 2024. Market Research Report.
- [2] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, “Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm,” in *Proceedings of the 2007 Conference on Analysis of Algorithms*, pp. 127–146, 2007.
- [3] A. Psaltis, *Streaming Data: Understanding the real-time pipeline*. Manning Publications, 2017.
- [4] S. Heule, M. Nunkesser, and A. Hall, “Hyperloglog in practice: Algorithmic engineering of a state of the art cardinality estimation algorithm,” in *Proceedings of the EDBT/ICDT 2013 Joint Conference*, pp. 683–692, Google Inc., 2013.
- [5] J. Kreps, N. Narkhede, J. Rao, *et al.*, “Kafka: a distributed messaging system for log processing,” in *Proceedings of the NetDB*, vol. 11, pp. 1–7, 2011.
- [6] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, A. Ghodsi, I. Stoica, and M. Zaharia, “Structured streaming: A declarative api for real-time applications in apache spark,” in *Proceedings of the 2018 International Conference on Management of Data*, pp. 601–613, 2018.

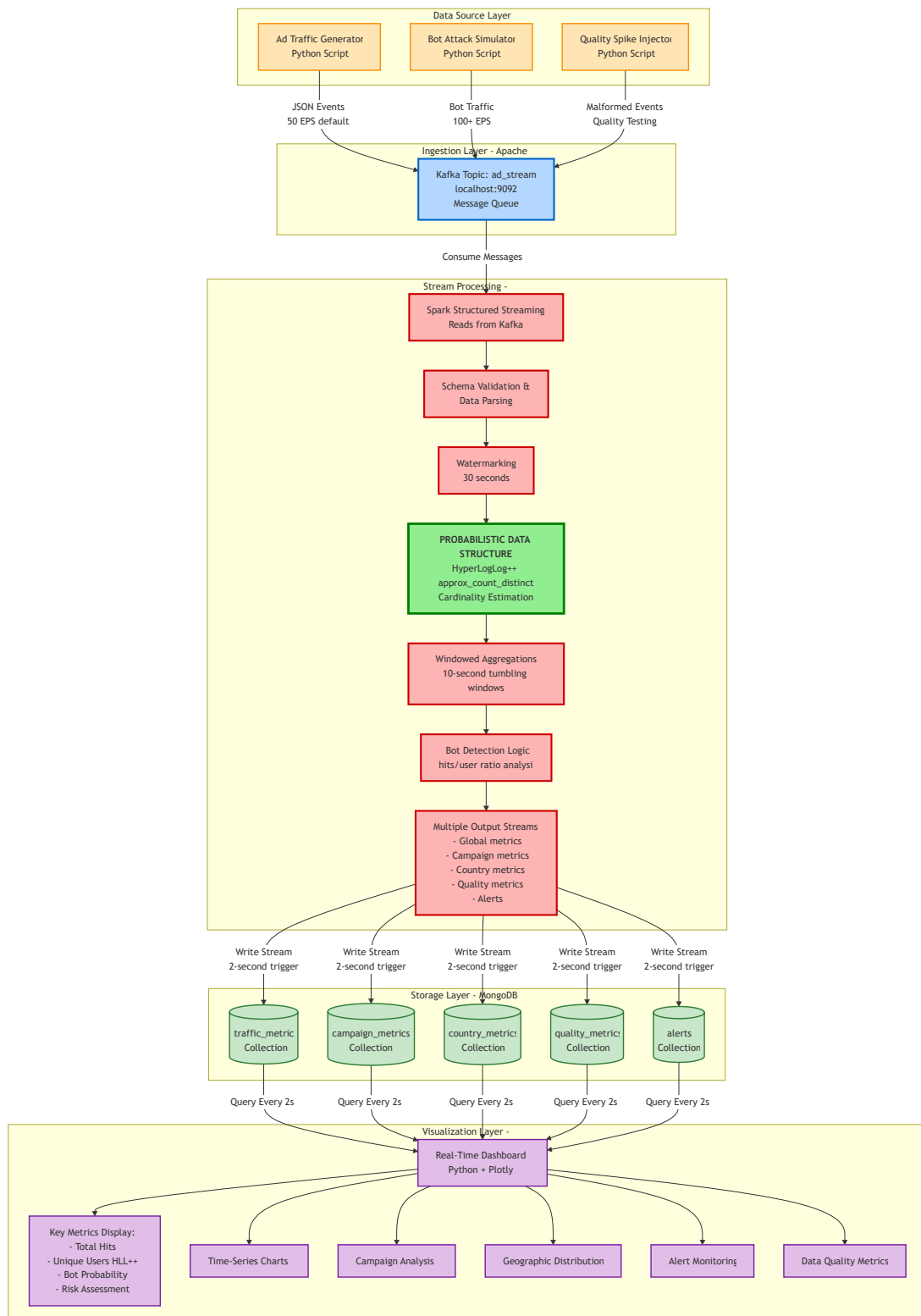


Figure 8. End-to-End System Architecture Pipeline